

SYSTEM ARCHITECTURE DOCUMENT

Binectics

Comprehensive Backend Development Guide

Derived from the production frontend — 155 routes, 40+ API endpoint groups, 5 user roles

Version: 2.0

Date: 26 May 2026

Author: Engineering Team

Classification: Internal — Engineering

Frontend Stack: Next.js 16, React 19, Tailwind CSS 4, TypeScript 5

Backend Target: NestJS 11 + MongoDB + Azure

API Base URL: `https://binectics-gym-dev-api-*.azurewebsites.net/api/v1`

Table of Contents

1. System Overview & Architecture Diagram

2. User Roles & Access Control

3. Data Entities & Relationships

4. Enums & Domain Constants

5. Authentication & Session Management

6. API Endpoints — Full Specification

7. Payment Processing & Multi-Currency

8. Real-Time Features (SSE / WebSocket)

9. File Upload & Storage

10. Notification System

11. Search & Discovery

12. Admin Panel Requirements

13. Dashboard Data Contracts

14. Form Data Shapes

15. Route Protection Matrix

16. Third-Party Integrations

17. Environment Variables

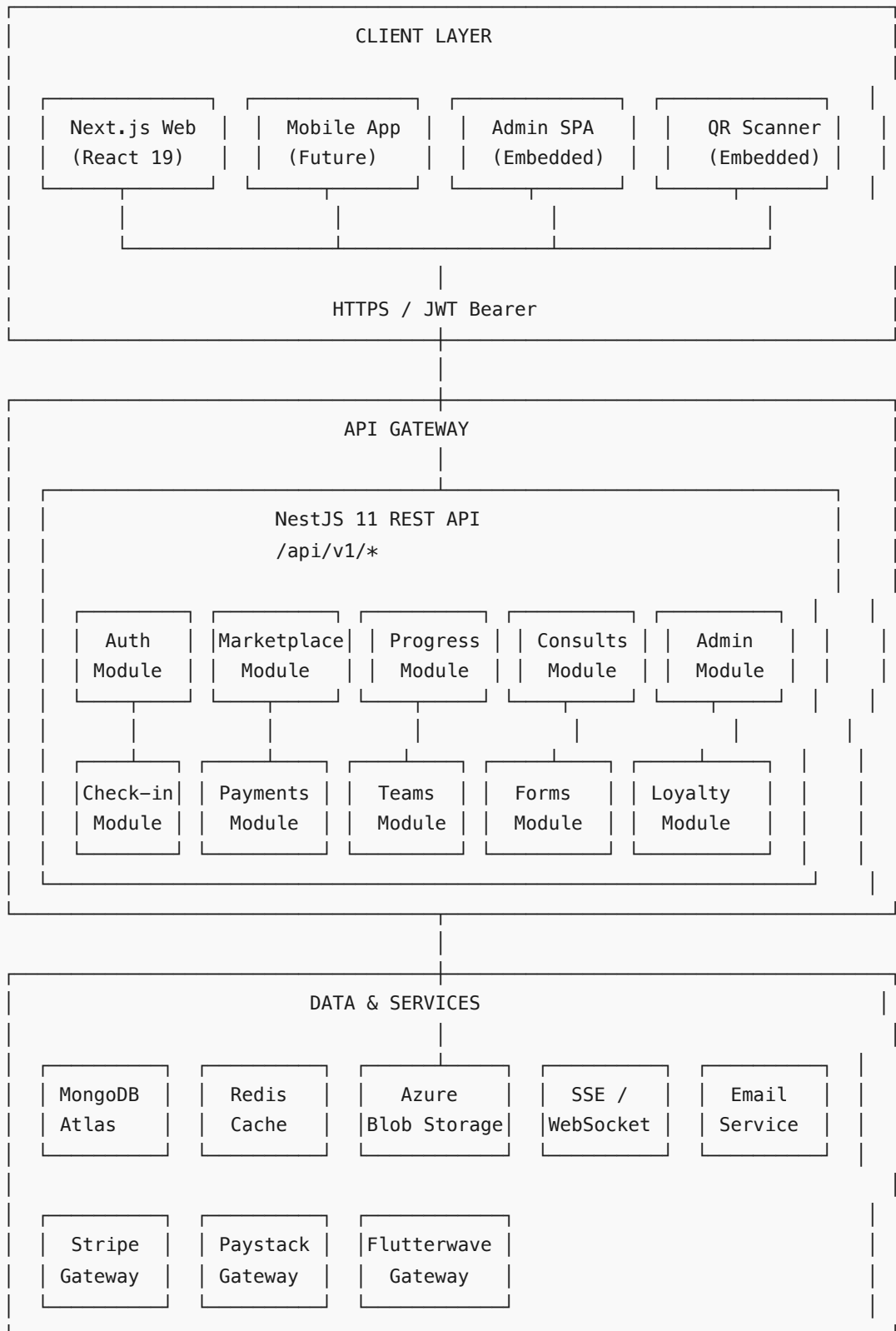
18. Deployment & Infrastructure

1. System Overview & Architecture Diagram

1.1 Platform Summary

Binectics is a global fitness marketplace connecting gym owners, personal trainers, dietitians, and fitness members. The platform operates across 50+ countries, supports 8 currencies, and processes payments through three gateways (Stripe, Paystack, Flutterwave).

1.2 High-Level Architecture



1.3 Module Breakdown

MODULE	RESPONSIBILITY	KEY ENTITIES
Auth	Registration, login, JWT, 2FA, password reset, email verification, OTP	User, Session, RefreshToken
Marketplace	Listings, plans, subscriptions, reviews, requests, documents	Listing, Plan, Subscription, Review, Request
Progress	Client management, journals, weight/meal/workout logs, diet plans	Client, Journal, WeightLog, MealLog, WorkoutLog, DietPlan
Consultations	Booking, availability, time slots, consultation types	ConsultationType, Availability, Booking
Check-in	QR scanning, attendance tracking, gym check-in	CheckIn
Payments	Multi-gateway routing, reconciliation, refunds, payouts	Payment, Payout, Refund
Teams	Organizations, members, roles, invitations	Organization, TeamMember, Invitation, Role
Forms	Intake forms, questions, responses, analytics	Form, Question, Response
Loyalty	Points, transactions, rewards, redemptions	PointsBalance, Transaction, Reward, Redemption
Notifications	In-app, email, SSE streaming, preferences	Notification, Preference
Admin	User management, listing moderation, disputes, fraud, compliance, feature flags	Dispute, FraudCase, FeatureFlag, AuditLog

2. User Roles & Access Control

2.1 Role Definitions

ROLE	ENUM VALUE	DASHBOARD PATH	CAPABILITIES
Fitness Member	USER	/dashboard/member	Browse marketplace, subscribe, book sessions, check-in, log health data, message providers
Gym Owner	GYM_OWNER	/dashboard/gym-owner	Manage gym listing, members, staff, schedules, plans, revenue, payouts, devices, locations
Personal Trainer	TRAINER	/dashboard/trainer	Manage clients, sessions, packages, earnings, profile, workouts, messaging
Dietitian	DIETITIAN	/dashboard/dietitian	Manage clients, consultations, meal plans, food DB, protocols, earnings, messaging
Admin	ADMIN	/admin/dashboard	Full platform management: users, listings, disputes, fraud, compliance, feature flags, analytics

2.2 Route Protection Matrix

ROUTE PATTERN	AUTH REQUIRED	ALLOWED ROLES	SPECIAL RULES
/dashboard/gym-owner/*	Yes	GYM_OWNER	—
/dashboard/trainer/*	Yes	TRAINER	—
/dashboard/dietitian/*	Yes	DIETITIAN	—
/dashboard/member/*	Yes	USER	—
/dashboard/* (shared)	Yes	All authenticated	Messages, bookings, settings, calendar, notifications
/admin/*	Yes	ADMIN	must_change_password flag forces redirect to /admin/change-password
/checkout/*	Yes	All authenticated	—
/check-in/*	Yes	USER	Active subscription required
/marketplace/*	No	Public	Auth optional for personalization

/login, /register/*	No	Unauthenticated only	Redirect to dashboard if already logged in
---------------------	----	----------------------	--

2.3 Team Permissions (RBAC)

Organizations (gyms, practices) support team members with granular permissions:

Team Permissions

- team:view_members
- team:invite_member
- team:remove_member
- team:update_member_role
- team:deactivate_member
- team:manage_roles
- team:manage_organization
- team:cancel_invitation
- team:view_invitations

Progress Permissions

- progress:view
- progress:create
- progress:edit
- progress:delete
- progress:manage_clients
- progress:invite_client

3. Data Entities & Relationships

3.1 Entity Relationship Overview

```

User ————— 1:1 ——— MarketplaceListing
|
|— 1:N — MembershipSubscription   |— 1:N — MembershipPlan
|— 1:N — CheckIn                 |— 1:N — Review
|— 1:N — ConsultationBooking     |— 1:N — MarketplaceRequest
|— 1:N — Notification            |— 1:N — Document
|— 1:N — WeightLog
|— 1:N — MealLog
|— 1:N — WorkoutLog
|— 1:N — JournalEntry
|— M:N — Organization (via TeamMember)
|
Organization — 1:N — TeamMember
                — 1:N — CheckIn
                — 1:N — AssignmentRule
                — 1:N — Facility

ConsultationType — 1:N — ConsultationBooking
AvailabilityRule — belongs to — User (provider)
DietPlan — belongs to — User (dietitian) — assigned to — User (client)
LoyaltyBalance — belongs to — User
LoyaltyReward — 1:N — LoyaltyRedemption
Form — 1:N — Question — 1:N — Response
FeatureFlag (admin-managed, standalone)
FraudCase — linked to — User
Dispute — links — User (member) + User (provider)

```


3.2 Core Entity Schemas

User

```
{
  _id: ObjectId,
  email: string (unique, indexed),
  password_hash: string,
  first_name: string,
  last_name: string,
  phone_number?: string,
  profile_picture?: string (URL),
  role: UserRole,                                // USER | GYM_OWNER | TRAINER | DIETITIAN |
ADMIN
  country_code?: string,                          // ISO 3166-1 alpha-2
  is_email_verified: boolean,
  is_onboarding_complete?: boolean,
  is_suspended: boolean,
  is_company: boolean,
  company_name?: string,
  company_logo?: string,
  fitness_goals?: string[],
  preferred_activities?: string[],
  date_of_birth?: Date,
  must_change_password: boolean,                  // For seeded admin accounts
  created_at: Date,
  updated_at: Date
}
Indexes: email (unique), role, country_code, is_suspended
```

MarketplaceListing

```
{
  _id: ObjectId,
  professional_id: ObjectId (ref: User),    // The provider who owns this listing
  organization_id?: ObjectId (ref: Organization),
  account_type: AccountType,                // gym_owner | personal_trainer | dietitian
  headline: string,
  bio: string,
  specialties: string[],
  certifications: string[],
  languages: string[],
  facilities: string[],
  amenities: string[],                      // AmenityKey enum values
  photos: string[],                         // URLs, ordered
  city?: string,
  country_code?: string,
  address?: string,
  contact_phone?: string,
  contact_email?: string,
  location?: {                              // GeoJSON Point
    type: "Point",
    coordinates: [number, number]           // [lng, lat]
  },
  currency: string,                         // ISO 4217
  price_from?: number,                     // Lowest plan price for display
  accepting_clients: boolean,
  is_published: boolean,
  verification_badge: VerificationBadge,    // NONE | VERIFIED | PREMIUM_VERIFIED |
FEATURED
  active_client_count: number,
  average_rating: number,                   // Computed, 0.00–5.00
  review_count: number,                     // Computed
  created_at: Date,
  updated_at: Date
}
Indexes: account_type, country_code, city, is_published, location (2dsphere),
         average_rating (desc), verification_badge
```

MembershipPlan

```
{
  _id: ObjectId,
  organization_id: ObjectId,
  listing_id?: ObjectId,
  created_by: ObjectId (ref: User),
  name: string,
  description?: string,
  plan_type: MembershipPlanType,           // SUBSCRIPTION | ONE_TIME
  duration_days: number,
  price: number,
  currency: string,
  features: string[],
  is_active: boolean,
  is_public: boolean,
  active_members: number,                  // Computed
  max_clients?: number,                   // For trainer/dietitian plans
  created_at: Date,
  updated_at: Date
}
```

MembershipSubscription

```
{
  _id: ObjectId,
  organization_id: ObjectId,
  plan_id: ObjectId (ref: MembershipPlan),
  member_user_id: ObjectId (ref: User),
  status: SubscriptionStatus,             // PENDING_PAYMENT | ACTIVE | EXPIRED |
  CANCELLED
  start_date: Date,
  end_date?: Date,
  amount_paid: number,
  currency: string,
  auto_renew: boolean,
  payment_reference?: string,
  payment_gateway?: PaymentGateway,
  created_at: Date,
  updated_at: Date
}
Indexes: member_user_id, organization_id, status, end_date
```

CheckIn

```
{
  _id: ObjectId,
  organization_id: ObjectId,
  listing_id: ObjectId,
  member_user_id: ObjectId (ref: User),
  subscription_id?: ObjectId,
  checked_in_at: Date,
  note?: string
}
Indexes: organization_id + checked_in_at, member_user_id + checked_in_at
```

ConsultationBooking

```
{
  _id: ObjectId,
  client_user_id: ObjectId (ref: User),
  provider_id: ObjectId (ref: User),
  consultation_type_id: ObjectId,
  starts_at: Date,
  ends_at: Date,
  status: BookingStatus, // PENDING | CONFIRMED | CANCELLED |
COMPLETED | NO_SHOW
  notes?: string,
  completion_note?: string,
  cancelled_by?: CancelledBy, // CLIENT | PROVIDER | ADMIN
  session_format?: string, // in-person | online
  add_ons?: string[],
  client_timezone: string,
  created_at: Date,
  updated_at: Date
}
Indexes: provider_id + starts_at, client_user_id + starts_at, status
```

Review

```
{
  _id: ObjectId,
  reviewer_id: ObjectId (ref: User),
  listing_id: ObjectId (ref: MarketplaceListing),
  target_type: ReviewTargetType,           // GYM | TRAINER | DIETITIAN
  rating: number,                          // 1-5
  comment?: string,
  tags?: string[],                         // "On time", "Personalised", etc.
  show_name: boolean,
  status: ReviewStatus,                    // VISIBLE | HIDDEN | REMOVED
  moderation_flags?: string[],
  created_at: Date,
  updated_at: Date
}
```

Organization

```
{
  _id: ObjectId,
  owner_id: ObjectId (ref: User),
  name: string,
  type: string,                            // gym | practice
  billing_email?: string,
  billing_address?: object,
  created_at: Date,
  updated_at: Date
}
```

WeightLog / MealLog / WorkoutLog / JournalEntry

```
// WeightLog
{ _id, user_id, weight_kg: number, date: Date, note?: string }

// MealLog
{ _id, user_id, meal_type: MealType, rating: MealRating,
  notes?: string, foods?: string[], date: Date }

// WorkoutLog
{ _id, user_id, activity_type: ActivityType, duration_minutes: number,
  intensity?: string, notes?: string, date: Date, calories_burned?: number }

// JournalEntry
{ _id, provider_id, client_user_id, plan_id?,
  date: Date, notes: string, weight?: number,
  mood?: ClientJournalMood, adherence_score?: number }
```

DietPlan

```
{
  _id: ObjectId,
  created_by: ObjectId (ref: User, dietitian),
  assigned_to?: ObjectId (ref: User, client),
  name: string,
  description?: string,
  category: string, // cutting | clinical | sport | maintenance
  meals_per_day: number,
  kcal_target?: number,
  macros?: { protein: number, carbs: number, fat: number },
  meal_slots: [{
    slot: MealSlot,
    description?: string,
    foods?: string[]
  }],
  delivery_type: DietPlanDeliveryType, // PLATFORM | DOCUMENT
  is_template: boolean,
  duration_weeks?: number,
  created_at: Date,
  updated_at: Date
}
```

Notification

```
{
  _id: ObjectId,
  user_id: ObjectId (ref: User),
  type: NotificationType,           // See enum section
  title: string,
  message: string,
  is_read: boolean,
  reference_type?: string,          // booking | subscription | review | ...
  reference_id?: string,
  created_at: Date
}
Indexes: user_id + is_read + created_at (desc)
```

Dispute

```
{
  _id: ObjectId,
  dispute_id: string,               // DSP-XXXX human-readable
  member_id: ObjectId (ref: User),
  provider_id: ObjectId (ref: User),
  reason: string,
  amount: number,
  currency: string,
  status: DisputeStatus,           // OPEN | ESCALATED | RESOLVED
  sla_deadline?: Date,
  resolution_message?: string,
  resolved_by?: ObjectId (ref: User),
  opened_at: Date,
  resolved_at?: Date
}
```

FraudCase

```
{
  _id: ObjectId,
  fraud_id: string,                      // FRD-XXXX-XXXXX
  user_id: ObjectId (ref: User),
  type: string,                          // refund_abuse | velocity_fraud |
chargeback_pattern | credential_misuse | ip_anomaly | promo_abuse
  signal: string,                        // Human-readable description
  linked_accounts: number,
  exposure_amount?: number,
  currency?: string,
  risk_level: RiskLevel,                 // HIGH | MEDIUM | WATCH | CLEARED
  flagged_at: Date,
  resolved_at?: Date
}
```

FeatureFlag

```
{
  _id: ObjectId,
  key: string (unique),
  description: string,
  status: FlagStatus,                   // ON | PARTIAL | OFF | KILLED
  rollout_percentage: number,           // 0-100
  rollout_segments?: string[],
  owner: string,
  section?: string,                     // growth | infrastructure | trust |
product
  metric_value?: string,
  created_at: Date,
  last_toggled_at?: Date
}
```


LoyaltyPointsBalance & Transaction

```
// Balance
{ _id, user_id: ObjectId, total_points: number, lifetime_earned: number }

// Transaction
{ _id, user_id: ObjectId, organization_id?: ObjectId,
  points: number, balance_after: number,
  event_type: LoyaltyEventType,
  reference_type?: string, reference_id?: string,
  metadata?: object, created_at: Date }

// Reward
{ _id, name: string, description: string,
  points_required: number, is_active: boolean }

// Redemption
{ _id, user_id: ObjectId, reward_id: ObjectId,
  points_spent: number, status: RedemptionStatus,
  created_at: Date }
```

Facility (Gym Equipment/Amenity Items)

```
{
  _id: ObjectId,
  organization_id: ObjectId,
  name: string,
  category: FacilityCategory,           // EQUIPMENT | ROOM | AMENITY | SERVICE
  status: FacilityStatus,               // AVAILABLE | MAINTENANCE
  condition: FacilityCondition,         // EXCELLENT | GOOD | FAIR
  description: string,
  image_url?: string,
  icon_key?: string,
  is_featured: boolean
}
```

AssignmentRule (Auto-assign clients to staff)

```

{
  _id: ObjectId,
  organization_id: ObjectId,
  name: string,
  description?: string,
  is_active: boolean,
  priority: number,
  min_amount?: number,
  plan_ids: ObjectId[],
  client_tiers: ClientTier[],           // STANDARD | VIP | PREMIUM
  strategy: AssignmentStrategy,         // ROUND_ROBIN | FIRST_AVAILABLE |
  LEAST_LOADED
  staff_user_ids: ObjectId[],
  last_assigned_index: number
}

```

Message / Conversation

```

// Conversation
{ _id, participants: ObjectId[], last_message_at: Date,
  unread_counts: { [userId]: number } }

// Message
{ _id, conversation_id: ObjectId, sender_id: ObjectId,
  body: string, attachments?: [{ type, url, name, size }],
  is_read: boolean, created_at: Date }

```

4. Enums & Domain Constants

All enums used by the frontend. The backend must accept and return these exact values.

4.1 User & Account

ENUM	VALUES
UserRole	USER, GYM_OWNER, TRAINER, DIETITIAN, ADMIN
AccountType	gym_owner, personal_trainer, dietitian, fitness_member
MemberStatus	PENDING, ACTIVE, INACTIVE
InvitationStatus	PENDING, ACCEPTED, EXPIRED, CANCELLED

4.2 Marketplace

ENUM	VALUES
VerificationBadge	NONE, VERIFIED, PREMIUM_VERIFIED, FEATURED
MembershipPlanType	SUBSCRIPTION, ONE_TIME
SubscriptionStatus	PENDING_PAYMENT, ACTIVE, EXPIRED, CANCELLED
PlanStatus	ACTIVE, INACTIVE, ARCHIVED

4.3 Booking & Consultations

ENUM	VALUES
BookingStatus	PENDING, CONFIRMED, CANCELLED, COMPLETED, NO_SHOW
CancelledBy	CLIENT, PROVIDER, ADMIN
ProviderRole	DIETITIAN, PERSONAL_TRAINER, OTHER
ExceptionType	UNAVAILABLE, CUSTOM_HOURS

4.4 Health & Progress

ENUM	VALUES
MealType	BREAKFAST, LUNCH, DINNER, SNACK

MealSlot	BREAKFAST, MORNING_SNACK, LUNCH, AFTERNOON_SNACK, DINNER, EVENING_SNACK
MealRating	GREAT, GOOD, OKAY, POOR
ActivityType	CARDIO, STRENGTH, FLEXIBILITY, HIIT, YOGA, SWIMMING, CYCLING, RUNNING, WALKING, OTHER
ClientJournalMood	EXCELLENT, GOOD, OKAY, LOW, STRESSED
DifficultyLevel	BEGINNER, INTERMEDIATE, ADVANCED
DietPlanDeliveryType	PLATFORM, DOCUMENT
RecommendCategory	RECOVERY, HYDRATION, NUTRITION, LIFESTYLE, EXERCISE, GENERAL

4.5 Reviews

ENUM	VALUES
ReviewTargetType	GYM, TRAINER, DIETITIAN
ReviewStatus	VISIBLE, HIDDEN, REMOVED

4.6 Payments

ENUM	VALUES
PaymentGateway	STRIPE, FLUTTERWAVE, PAYSTACK
CheckInPeriod	TODAY, WEEK, MONTH

4.7 Facilities

ENUM	VALUES
FacilityCategory	EQUIPMENT, ROOM, AMENITY, SERVICE
FacilityStatus	AVAILABLE, MAINTENANCE
FacilityCondition	EXCELLENT, GOOD, FAIR
AmenityKey	WIFI, TOWEL_SERVICE, PERSONAL_TRAINING, GROUP_CLASSES, CHILDCARE, ACCESS_24_7, LOCKER_ROOMS, SHOWERS, PARKING, JUICE_BAR, SAUNA, POOL, STEAM_ROOM

4.8 Loyalty

ENUM	VALUES
LoyaltyEventType	SUBSCRIPTION_PURCHASE, GYM_CHECK_IN, JOURNAL_LOGGED, REWARD_REDEMPTION, ADMIN_ADJUSTMENT, SIGNUP_BONUS
RedemptionStatus	PENDING, FULFILLED, CANCELLED

4.9 Team & Assignment

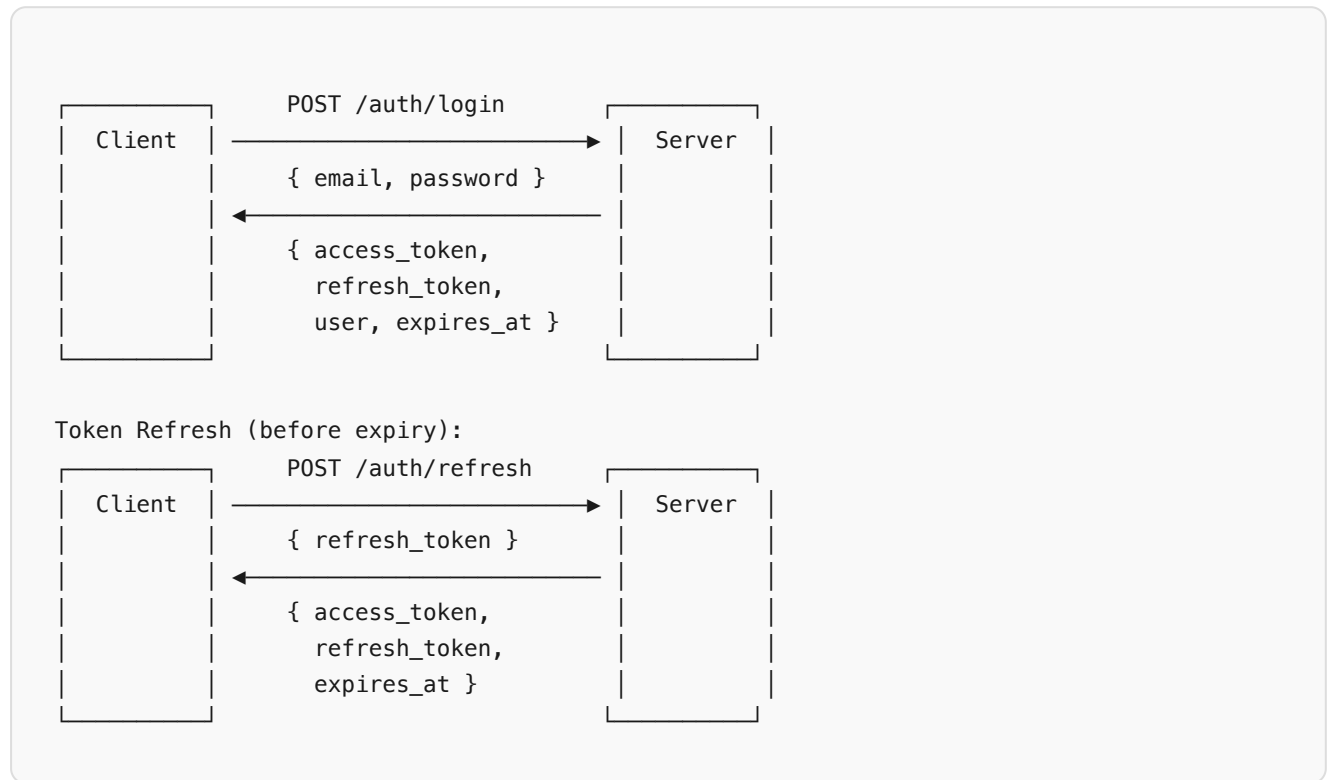
ENUM	VALUES
ClientTier	STANDARD, VIP, PREMIUM
AssignmentStrategy	ROUND_ROBIN, FIRST_AVAILABLE, LEAST_LOADED

4.10 Notifications

ENUM	VALUES
NotificationType	BOOKING_CREATED, BOOKING_CONFIRMED, BOOKING_CANCELLED, BOOKING_RESCHEDULED, BOOKING_COMPLETED, BOOKING_REMINDER, CLIENT_INVITATION, CLIENT_REQUEST, CLIENT_ACCEPTED, CLIENT_DEPARTED, MARKETPLACE_REQUEST_RECEIVED, MARKETPLACE_REQUEST_ACCEPTED, MARKETPLACE_REQUEST_REJECTED, REVIEW_RECEIVED, REVIEW_RESPONSE, DIET_PLAN_ASSIGNED, WORKOUT_PLAN_ASSIGNED, SUBSCRIPTION_CREATED, SUBSCRIPTION_EXPIRING, SUBSCRIPTION_EXPIRED, PAYMENT_RECEIVED, VERIFICATION_APPROVED, VERIFICATION_REJECTED, SYSTEM_ANNOUNCEMENT, ACCOUNT_SUSPENDED

5. Authentication & Session Management

5.1 Authentication Flow



5.2 Token Strategy

TOKEN	LIFETIME	STORAGE	USAGE
Access Token (JWT)	60 minutes	localStorage + cookie	Authorization: Bearer {token} on all API requests
Refresh Token	30 days	localStorage + cookie	Auto-refresh 5 min before expiry; used on 401 retry

5.3 Auth Endpoints

POST	<code>/auth/register</code>	Create account (firstName, lastName, email, password, role)
POST	<code>/auth/login</code>	Email/password login → tokens + user object
POST	<code>/auth/logout</code>	Invalidate refresh token
POST	<code>/auth/refresh</code>	Exchange refresh token for new token pair
POST	<code>/auth/verify-otp</code>	Verify OTP code after registration

POST	<code>/auth/resend-otp</code>	Resend OTP code
POST	<code>/auth/verify-email</code>	Email verification via token link
POST	<code>/auth/forgot-password</code>	Send password reset email
POST	<code>/auth/reset-password</code>	Reset password with token
POST	<code>/auth/change-password</code>	Change password (authenticated, for <code>must_change_password</code> flow)
PATCH	<code>/auth/profile</code>	Update user profile fields
PATCH	<code>/auth/profile/picture</code>	Upload profile picture (multipart)

5.4 Session Management

- **Auto-refresh:** Client monitors token expiry and auto-refreshes 5 minutes before expiration
- **Session modal:** When token expires without refresh, a modal prompts user to continue or logout
- **401 retry:** On any 401 response, client attempts one automatic refresh + re-request
- **must_change_password:** Seeded admin accounts have this flag. Middleware forces redirect to `/admin/change-password` until the password is changed.
- **2FA:** Optional 6-digit OTP challenge during login. Trust-device option available.

5.5 Registration Flow

```

Register → POST /auth/register → Redirect to /verification
      |
      Enter OTP code
      |
      POST /auth/verify-otp
      |
      Redirect to /login
      |
      POST /auth/login → Dashboard
      |
      (If onboarding incomplete)
      |
      Redirect to /onboarding/{role}
  
```

6. API Endpoints — Full Specification

All endpoints prefixed with `/api/v1`. All authenticated endpoints require `Authorization: Bearer {access_token}`. Standard response wrapper: `{ success: boolean, data?: T, message?: string, errors?: object }`

6.1 Marketplace & Listings

GET	/marketplace/listings	Search listings (query: account_type, country_code, city, specialties, price_min, price_max, rating_min, currency, page, limit, sort)
GET	/marketplace/listings/{id}	Get listing detail (populated: professional, plans, reviews summary)
GET	/marketplace/featured	Featured listings for homepage
GET	/marketplace/my-listing	Current provider's own listing (auth required)
POST	/marketplace/listings	Create listing (provider only)
PATCH	/marketplace/listings	Update listing (owner only)
POST	/marketplace/listings/{id}/publish	Publish/unpublish listing

6.2 Plans & Subscriptions

GET	/marketplace/plans/{listingId}	Get plans for a listing (public)
POST	/marketplace/plans	Create plan (provider)
PATCH	/marketplace/plans/{planId}	Update plan
POST	/marketplace/subscriptions	Create subscription (after payment)
GET	/marketplace/subscriptions/my-subscriptions	User's active subscriptions
PATCH	/marketplace/subscriptions/{id}/renew	Renew subscription
DELETE	/marketplace/subscriptions/{id}	Cancel subscription

6.3 Requests & Reviews

POST	/marketplace/requests/{listingId}	Send connection/inquiry request to provider
GET	/marketplace/requests	View sent requests
GET	/marketplace/requests/{listingId}/received	View received requests (provider)
PATCH	/marketplace/requests/{id}	Accept/reject request
POST	/marketplace/reviews/{listingId}	Leave review (rating, comment, tags)

GET /marketplace/reviews/{listingId} Get reviews for listing (paginated)

6.4 Check-In

POST /checkins/scan QR code check-in (listing_id, note)

GET /checkins/my-status Current check-in status for member

GET /checkins/my-history Member check-in history (query: period=TODAY|WEEK|MONTH)

GET /checkins/dashboard-stats Personal check-in stats (streak, counts)

GET /checkins/organizations/{orgId} Org check-in history (gym owner)

GET /checkins/organizations/{orgId}/dashboard-stats Org check-in stats (gym owner)

6.5 Consultations

GET /consultations/types Available consultation types

POST /consultations/types Create consultation type (provider)

GET /consultations/provider/availability Provider's availability rules

POST /consultations/provider/availability Set availability rules

GET /consultations/slots Available time slots (query: provider_id, date_range)

GET /consultations/bookings List consultation bookings

POST /consultations/bookings Book consultation

PATCH /consultations/bookings/{id} Update booking status

6.6 Progress Tracking

GET /progress/clients Provider's client list

GET /progress/clients/{clientId} Client detail with logs

POST /progress/requests Invite client

GET /progress/weight Weight log entries

POST /progress/weight Add weight entry

GET /progress/meals Meal feedback history

POST /progress/meals Log meal

GET /progress/activities Activity reports

POST /progress/activities Log activity

GET /progress/journals Journal entries (provider view)

POST	/progress/journals	Create journal entry
GET	/progress/diet-plans	Assigned diet plans (client view)
POST	/progress/diet-plans	Create diet plan (dietitian)
PATCH	/progress/diet-plans/{id}	Update diet plan
GET	/progress/dashboard-stats	Progress dashboard summary

6.7 Teams & Organizations

GET	/teams/organizations	User's organizations
POST	/teams/organizations	Create organization
PATCH	/teams/organizations/{orgId}	Update organization
GET	/teams/organizations/{orgId}/members	List team members
POST	/teams/invitations	Send team invite
PATCH	/teams/invitations/{id}/accept	Accept invite
PATCH	/teams/members/{id}/role	Update member role
DELETE	/teams/members/{id}	Remove team member

6.8 Notifications

GET	/notifications	List notifications (query: is_read, type, page, limit)
GET	/notifications/unread-count	Unread count for badge
SSE	/notifications/stream	Real-time notification stream (query: token)
PATCH	/notifications/{id}/read	Mark as read
PATCH	/notifications/read-all	Mark all as read
GET	/notifications/preferences	Get notification preferences
PATCH	/notifications/preferences	Update notification preferences

6.9 Messages

GET	/messages/conversations	List conversations with last message preview
GET	/messages/conversations/{id}	Full conversation thread
POST	/messages/send	Send message (conversationId, body, attachments)
POST	/messages/upload	Upload file attachment (multipart)

6.10 Forms

GET	/forms	List provider's forms
POST	/forms	Create intake form
GET	/forms/{id}/questions	Get form questions
POST	/forms/{id}/responses	Submit form response (client)
GET	/forms/{id}/analytics	Response analytics

6.11 Loyalty

GET	/loyalty/balance	User's points balance
GET	/loyalty/transactions	Points transaction history
GET	/loyalty/rewards	Available rewards
POST	/loyalty/redemptions	Redeem points for reward

6.12 Admin

GET	/admin/users	User list (query: role, status, search, page, limit)
GET	/admin/users/{userId}	User detail with notes
PATCH	/admin/users/{userId}/suspend	Suspend user account
PATCH	/admin/users/{userId}/unsuspend	Unsuspend user
GET	/admin/metrics	Platform-wide metrics dashboard
GET	/admin/listings	Pending listings for review
PATCH	/admin/listings/{id}/approve	Approve listing
PATCH	/admin/listings/{id}/reject	Reject listing (with reason)
GET	/admin/disputes	List disputes
PATCH	/admin/disputes/{id}/resolve	Resolve dispute
GET	/admin/feature-flags	List feature flags
POST	/admin/feature-flags	Create feature flag
PATCH	/admin/feature-flags/{id}	Update feature flag
POST	/admin/loyalty/points	Admin adjust user points

6.13 Utility

GET	/utility/countries	List all supported countries
-----	--------------------	------------------------------

GET	/utility/currencies	List supported currencies
GET	/utility/timezones	List timezones
POST	/utility/feedback	Submit user feedback

7. Payment Processing & Multi-Currency

7.1 Gateway Routing Logic

Payment gateway selection is automatic based on the member's currency and country:

CURRENCY	PRIMARY GATEWAY	FALLBACK	REGION
NGN (Nigerian Naira)	Paystack	Flutterwave	Nigeria
GHS (Ghanaian Cedi)	Paystack	Flutterwave	Ghana
KES (Kenyan Shilling)	Paystack	Flutterwave	Kenya
ZAR (South African Rand)	Paystack	Flutterwave	South Africa
UGX, TZS, RWF, EGP, XOF, XAF	Flutterwave	—	Pan-African
USD, GBP, EUR, AED, etc.	Stripe	—	Global

7.2 Payment Flow

1. User selects plan → frontend determines gateway
2. Frontend initializes payment SDK (Stripe Elements / Paystack Popup / Flutterwave Modal)
3. User completes payment in SDK
4. SDK returns payment_reference
5. Frontend POST /marketplace/subscriptions { plan_id, payment_reference, gateway }
6. Backend verifies payment with gateway API
7. Backend creates MembershipSubscription (status: ACTIVE)
8. Backend triggers SUBSCRIPTION_CREATED notification

7.3 Reconciliation Requirements

- Unified ledger normalizing webhook formats from all 3 gateways
- 15-minute reconciliation loop flagging discrepancies
- Settlement timeline tracking per gateway (Stripe: T+2, Paystack: T+1, Flutterwave: T+1–3)
- Display-time currency conversion for discovery ("~\$12/month") — charge in provider's native currency
- Platform fee: 5% on all transactions (configurable per country)

7.4 Webhook Endpoints (Backend-to-Backend)

POST	/webhooks/stripe	Stripe webhook (payment_intent.succeeded, invoice.paid, charge.refunded, etc.)
POST	/webhooks/paystack	Paystack webhook (charge.success, transfer.success, refund.processed)
POST	/webhooks/flutterwave	Flutterwave webhook (charge.completed, transfer.completed)

8. Real-Time Features

8.1 Server-Sent Events (SSE)

The frontend connects to an SSE endpoint for real-time notifications:

```
GET /api/v1/notifications/stream?token={JWT}

Content-Type: text/event-stream
Connection: keep-alive

data: {"type":"BOOKING_CONFIRMED","title":"Booking confirmed","message":"Your session with Sarah Okafor is confirmed for Tuesday 10:00"}

data: {"type":"CLIENT_REQUEST","title":"New client request","message":"Tunde Adebayo wants to connect"}
```

8.2 Polling Fallback

If SSE connection fails, the frontend polls every 60 seconds:

```
GET /api/v1/notifications/unread-count → { count: 3 }
```

8.3 Real-Time Messaging

Messages page requires real-time delivery. Recommended approach:

- **Primary:** WebSocket or SSE channel per conversation
- **Events:** new_message, typing_indicator, read_receipt
- **Fallback:** Polling every 10 seconds when in active conversation

8.4 Live Activity Feed (Gym Dashboard)

The gym owner dashboard shows a live activity feed (check-ins, subscriptions, refunds). This should be powered by SSE or polling with a 30-second interval.

9. File Upload & Storage

9.1 Upload Requirements

FEATURE	FILE TYPES	MAX SIZE	ENDPOINT	STORAGE
Profile picture	JPEG, PNG	5 MB	PATCH /auth/profile/picture	Azure Blob
Listing photos (up to 8)	JPEG, PNG	10 MB each	POST /marketplace/documents/{id}	Azure Blob
Verification documents	PDF, JPEG, PNG	20 MB	POST /marketplace/documents/{id}	Azure Blob (private)
Message attachments	PDF, JPEG, PNG, MP4	25 MB	POST /messages/upload	Azure Blob
Facility item images	JPEG, PNG	5 MB	Via facility CRUD	Azure Blob

9.2 Image Processing

- Profile pictures: Resize to 256×256, generate thumbnail at 48×48
- Listing photos: Resize to max 1920px width, generate card thumbnail at 400×300
- Verification docs: Store as-is (private container, admin-only access)

10. Notification System

10.1 Notification Types & Triggers

TYPE	TRIGGER	CHANNELS	RECIPIENT
BOOKING_CREATED	Client books session	In-app, Email	Provider
BOOKING_CONFIRMED	Provider confirms booking	In-app, Email	Client
BOOKING_CANCELLED	Either party cancels	In-app, Email	Other party
BOOKING_REMINDER	24h before session	In-app, Email	Both
SUBSCRIPTION_CREATED	Payment confirmed	In-app, Email	Member + Provider
SUBSCRIPTION_EXPIRING	7 days before end_date	In-app, Email	Member
SUBSCRIPTION_EXPIRED	end_date passed	In-app, Email	Member
PAYMENT_RECEIVED	Payout processed	In-app, Email	Provider
REVIEW_RECEIVED	New review posted	In-app	Provider
CLIENT_INVITATION	Provider invites client	In-app, Email	Client
VERIFICATION_APPROVED	Admin approves listing	In-app, Email	Provider
VERIFICATION_REJECTED	Admin rejects listing	In-app, Email	Provider
SYSTEM_ANNOUNCEMENT	Admin broadcast	In-app	All / Segment
ACCOUNT_SUSPENDED	Admin suspends account	In-app, Email	User

10.2 User Preferences

Users can toggle each notification type for email and in-app independently. Stored per user.

11. Search & Discovery

11.1 Marketplace Search

PARAMETER	TYPE	DESCRIPTION
q	string	Full-text search on headline, bio, specialties
account_type	enum	gym_owner personal_trainer dietitian
country_code	string	ISO 3166-1 alpha-2
city	string	City name filter
specialties	string[]	Specialty filter (AND)
amenities	string[]	Amenity filter (AND)
price_min	number	Minimum price_from
price_max	number	Maximum price_from
rating_min	number	Minimum average_rating (e.g., 4.0)
currency	string	ISO 4217 currency code
lat, lng, radius	number	Geo-proximity search (km)
sort	string	rating_desc price_asc price_desc newest
page, limit	number	Pagination (default: page=1, limit=20)

11.2 Recommended Indexes

- Full-text index on headline , bio , specialties
- 2dsphere index on location for geo queries
- Compound index: { account_type, country_code, is_published, average_rating: -1 }
- Compound index: { account_type, country_code, price_from }

12. Dashboard Data Contracts

12.1 Admin Dashboard KPIs

The admin overview page displays these aggregated metrics. Backend should provide a single `GET /admin/metrics` endpoint or composable sub-endpoints:

KPI	SOURCE	COMPUTATION
GMV (30d)	Subscriptions	SUM(amount_paid) WHERE created_at > 30d ago
Active providers	Listings	COUNT WHERE is_published = true
Active members	Subscriptions	COUNT DISTINCT(member_user_id) WHERE status = ACTIVE
Pending listings	Listings	COUNT WHERE verification_badge = NONE AND is_published = true
Open disputes	Disputes	COUNT WHERE status = OPEN
Take rate	Payments	platform_fees / total_gmv × 100
Retention (90d)	Subscriptions	% users with active sub who had one 90d ago
Provider NPS	Feedback	Net Promoter Score from provider surveys

12.2 Gym Owner Dashboard KPIs

KPI	SOURCE	COMPUTATION
Revenue (30d)	Subscriptions	SUM(amount_paid) for org's plans, 30d window + 12-day sparkline
Active members	Subscriptions	COUNT WHERE status = ACTIVE for org
Check-ins today	CheckIns	COUNT WHERE org + today
Churn rate (30d)	Subscriptions	Cancelled or expired / total at start of period
MRR	Subscriptions	SUM(monthly normalized amount) for active subs
Avg LTV	Subscriptions	Avg total revenue per member
Retention (90d)	Subscriptions	% renewed after first period

12.3 Trainer Dashboard KPIs

KPI	SOURCE	COMPUTATION
-----	--------	-------------

Sessions this week	ConsultationBookings	COUNT WHERE provider + this week + status != CANCELLED
Active clients	Progress clients	COUNT active client relationships
MTD earnings	Payments	SUM(provider_amount) for current month
Rating (30d)	Reviews	AVG(rating) WHERE created_at > 30d ago

12.4 Dietitian Dashboard KPIs

KPI	SOURCE	COMPUTATION
Active clients	Progress clients	COUNT active client relationships
Adherence (30d avg)	Journals	AVG(adherence_score) for provider's clients, 30d
Plans expiring (7d)	DietPlans	COUNT WHERE end_date within 7d
MTD earnings	Payments	SUM(provider_amount) for current month

12.5 Member Dashboard KPIs

KPI	SOURCE	COMPUTATION
Current streak	CheckIns/Workouts	Consecutive days with activity
This week	Workouts	Completed / planned count
Next session	ConsultationBookings	Next upcoming confirmed booking
Current weight	WeightLog	Most recent entry

13. Form Data Shapes

Below are the exact request body shapes the backend must accept, derived from the frontend forms:

13.1 Registration

```
POST /auth/register
{
  "first_name": string,           // min 2, max 50
  "last_name": string,           // min 2, max 50
  "email": string,                // valid email
  "password": string,             // min 8-12 (role-dependent), requires uppercase + lowercase
  + digit + special
  "role": "USER" | "GYM_OWNER" | "TRAINER" | "DIETITIAN",
  "country_code?": string,       // ISO 3166-1 alpha-2
  "phone_number?": string,
  "accept_tos": boolean          // must be true
}
```

13.2 Marketplace Listing

```
POST/PATCH /marketplace/listings
{
  "headline": string,             // max 80 chars
  "bio": string,                  // min 50, max 500
  "specialties": string[],
  "certifications": string[],
  "languages": string[],
  "facilities": string[],
  "amenities": string[],          // AmenityKey values
  "photos": string[],             // URLs (ordered)
  "city": string,
  "country_code": string,
  "address": string,
  "contact_phone?": string,
  "contact_email?": string,
  "currency": string,
  "accepting_clients": boolean
}
```

13.3 Membership Plan

```
POST /marketplace/plans
{
  "name": string,
  "description"?: string,
  "plan_type": "SUBSCRIPTION" | "ONE_TIME",
  "duration_days": number,
  "price": number,
  "currency": string,           // ISO 4217
  "features": string[],
  "is_active": boolean,
  "is_public": boolean,
  "max_clients"?: number
}
```

13.4 Consultation Booking

```
POST /consultations/bookings
{
  "consultation_type_id": string,
  "provider_id": string,
  "starts_at": string,         // ISO 8601 datetime
  "ends_at": string,
  "client_timezone": string,
  "session_format"?: "in-person" | "online",
  "notes"?: string,
  "add_ons"?: string[]
}
```

13.5 Check-In

```
POST /checkins/scan
{
  "listing_id": string,
  "note"?: string
}
Response: { member_name, membership_type, streak_count, next_class?, photo_url? }
```

13.6 Review

```
POST /marketplace/reviews/{listingId}
{
  "rating": number,           // 1-5
  "comment"?: string,        // min 30, max 500
  "tags"?: string[],         // ["On time", "Personalised", ...]
  "show_name": boolean
}
```

13.7 Weight / Meal / Workout Log

```
POST /progress/weight    { "weight_kg": number, "date": string, "note"?: string }
POST /progress/meals     { "meal_type": MealType, "rating": MealRating, "foods"?: string[],
"notes"?: string, "date": string }
POST /progress/activities { "activity_type": ActivityType, "duration_minutes": number,
"intensity"?: string, "notes"?: string, "date": string }
```

13.8 Diet Plan

```
POST /progress/diet-plans
{
  "name": string,
  "description"?: string,
  "category": "cutting" | "clinical" | "sport" | "maintenance",
  "meals_per_day": number,
  "kcal_target"?: number,
  "macros"?: { "protein": number, "carbs": number, "fat": number },
  "meal_slots": [{ "slot": MealSlot, "description"?: string, "foods"?: string[] }],
  "delivery_type": "PLATFORM" | "DOCUMENT",
  "is_template": boolean,
  "assigned_to"?: string      // User ID of client
}
```

13.9 Dispute Resolution (Admin)

```
PATCH /admin/disputes/{id}/resolve
{
  "resolution_message": string, // min 20, max 2000
  "outcome": "refund_full" | "refund_partial" | "no_refund" | "credit"
}
```

14. Third-Party Integrations

SERVICE	PURPOSE	INTEGRATION TYPE	NOTES
Stripe	Card payments (global)	Server SDK + Webhooks	PaymentIntents API, Stripe Connect for provider payouts
Paystack	African payments (NGN, GHS, KES, ZAR)	Server SDK + Webhooks	Popup integration, bank transfer support
Flutterwave	Pan-African payments	Server SDK + Webhooks	14+ African currencies, USSD, mobile money
Azure Blob Storage	File/image storage	Server SDK	Separate containers for public assets vs private verification docs
Email Service	Transactional emails	SMTP / API	SendGrid or similar; templates for booking confirmations, password reset, etc.
Google Maps	Location services	Frontend JS SDK	Map view for marketplace, geocoding for address input
Google Analytics 4	Usage analytics	Frontend Script	Already integrated via <code>NEXT_PUBLIC_GOOGLE_ANALYTICS_ID</code>
Sentry	Error monitoring	Frontend SDK	Via <code>NEXT_PUBLIC_SENTRY_DSN</code>

15. Environment Variables

15.1 Frontend (NEXT_PUBLIC_*)

```
NEXT_PUBLIC_APP_URL=http://localhost:3001
NEXT_PUBLIC_API_URL=https://binectics-gym-dev-api-*.azurewebsites.net/api/v1
NEXT_PUBLIC_SESSION_TIMEOUT=3600000
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_xxx
NEXT_PUBLIC_FLUTTERWAVE_PUBLIC_KEY=FLWPUBK_TEST-xxx
NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY=pk_test_xxx
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=xxx
NEXT_PUBLIC_GOOGLE_ANALYTICS_ID=G-xxx
NEXT_PUBLIC_SENTRY_DSN=https://xxx@sentry.io/xxx
NEXT_PUBLIC_ENABLE_ANALYTICS=false
NEXT_PUBLIC_ENABLE_COOKIE_CONSENT=true
NEXT_PUBLIC_MAINTENANCE_MODE=false
```

15.2 Backend (Required — NOT in frontend)

```
# Database
MONGODB_URI=mongodb+srv://...
REDIS_URL=redis://...

# Authentication
JWT_SECRET=xxx
JWT_ACCESS_EXPIRY=3600           # seconds
JWT_REFRESH_EXPIRY=2592000       # 30 days
OTP_EXPIRY=300                   # 5 minutes

# Payment Secrets
STRIPE_SECRET_KEY=sk_test_xxx
STRIPE_WEBHOOK_SECRET=whsec_xxx
PAYSTACK_SECRET_KEY=sk_test_xxx
PAYSTACK_WEBHOOK_SECRET=xxx
FLUTTERWAVE_SECRET_KEY=FLWSECK_TEST-xxx
FLUTTERWAVE_WEBHOOK_HASH=xxx

# Storage
AZURE_STORAGE_CONNECTION_STRING=xxx
AZURE_BLOB_CONTAINER_PUBLIC=assets
AZURE_BLOB_CONTAINER_PRIVATE=verification-docs

# Email
SENDGRID_API_KEY=SG.xxx
EMAIL_FROM=noreply@binectics.com
SUPPORT_EMAIL=support@binectics.com

# External
GOOGLE_MAPS_SERVER_KEY=xxx
SENTRY_DSN=https://xxx@sentry.io/xxx
```

16. Deployment & Infrastructure

16.1 Architecture

COMPONENT	SERVICE	REGION
Frontend	Vercel (Next.js)	Edge (global)
Backend API	Azure App Service	Canada Central
Database	MongoDB Atlas	Multi-region
Cache	Azure Redis Cache	Canada Central
File Storage	Azure Blob Storage	Canada Central
CDN	Cloudflare	Global
DNS	Cloudflare	Global

16.2 Scaling Considerations

- **Horizontal scaling:** API is stateless (JWT-based), can scale horizontally behind load balancer
- **Database indexes:** Critical for marketplace search (geo, full-text, compound)
- **Rate limiting:** Frontend shows `/rate-limit` page — backend should return 429 with `Retry-After` header
- **Maintenance mode:** Frontend checks `NEXT_PUBLIC_MAINTENANCE_MODE` — backend should return 503 globally when in maintenance
- **Caching:** Redis for session data, notification counts, feature flags, and frequently accessed listings

16.3 Background Jobs

JOB	FREQUENCY	DESCRIPTION
Subscription expiry check	Every 1h	Find expiring subscriptions, send notifications, update status
Payment reconciliation	Every 15m	Verify gateway payments match internal ledger
Booking reminders	Every 30m	Send 24h pre-session reminders
Streak calculation	Daily (midnight)	Calculate member activity streaks
Churn risk scoring	Daily	Flag members with declining check-in frequency

Fraud signal scanning	Every 6h	Analyze refund patterns, velocity checks, IP anomalies
Analytics aggregation	Hourly	Pre-compute dashboard metrics for admin and provider dashboards

Appendix A: Complete Route Inventory (155 pages)

Authentication (10 routes)

ROUTE	TYPE	AUTH
/login	Client	No (redirect if authed)
/register	Client	No
/register/user	Client	No
/register/gym-owner	Client	No
/register/trainer	Client	No
/register/dietitian	Client	No
/forgot-password	Client	No
/reset-password/[token]	Client	No
/verification	Client	No
/verify-email/[token]	Client	No

Admin Dashboard (28 routes)

ROUTE	DESCRIPTION
/admin/dashboard	Overview with KPIs, queues, activity
/admin/users	User list with filters
/admin/users/[userId]	User detail
/admin/analytics	Platform analytics
/admin/disputes	Dispute list
/admin/disputes/[disputeId]	Dispute detail
/admin/disputes/[disputeId]/resolve	Resolution form
/admin/fraud	Fraud cases
/admin/fraud/[fraudId]	Fraud case detail
/admin/payments	Payment records

/admin/payments/[paymentId]	Payment detail
/admin/refunds	Refund processing
/admin/countries	Country management
/admin/countries/[countryCode]	Country detail
/admin/compliance	Compliance dashboard
/admin/feature-flags	Feature flag list
/admin/feature-flags/[flagId]	Flag configuration
/admin/listings	Listing moderation queue
/admin/listings/[listingId]	Listing review
/admin/listings/[listingId]/reject	Rejection form
/admin/reviews	Review moderation
/admin/reviews/[reviewId]	Review detail
/admin/team-roles	Admin team management
/admin/tickets	Support tickets
/admin/audit-log	Audit trail
/admin/change-password	Forced password change
/admin/create-super-admin	Super admin creation

Provider Dashboards (45 routes)

Gym Owner: 19 routes under /dashboard/gym-owner/* (overview, members, member detail, schedule, class detail, checkins, devices, locations, location detail, staff, staff detail, plans, revenue, payouts, facility, marketing, reviews, settings, tax)

Trainer: 13 routes under /dashboard/trainer/* (overview, clients, client detail, bulk-invite, sessions, session detail, workouts/create, plans, earnings, messages, profile, settings, tax)

Dietitian: 16 routes under /dashboard/dietitian/* (overview, consultations, clients, client detail, meal-plans, foods, food detail, protocols, calendar, earnings, messages, feedback, profile, settings, tax, pdf-preview)

Member Dashboard (9 routes)

/dashboard/member/* : home, health-metrics, weight-log, meal-log, workout-log, streaks, billing, profile, help

Shared Dashboard (8 routes)

/dashboard/* : messages, bookings, calendar, notifications, profile-edit, saved-providers, settings

Marketplace & Booking (13 routes)

/marketplace , /marketplace/[listingId] , /marketplace/search , /marketplace/map ,
 /marketplace/compare , /booking , /booking/[id] , /booking/[id]/receipt ,
 /booking/confirmed , /booking/recurring , /checkout , /checkout/success ,
 /checkout/cancelled

Public & Marketing (22 routes)

Landing, about, blog, blog/[slug], careers, case-studies, contact, for-gyms, for-trainers, for-dietitians, for-members, help, partners, press, pricing, privacy, terms, cookies, qr-help, status, support, legal

Onboarding (4 routes)

/onboarding/gym-owner , /onboarding/trainer , /onboarding/dietitian , /onboarding/member

Utility & Error (12 routes)

check-in/[gymId], review/[providerId], verification, teams/invite/accept, 2fa-recovery, account-deleted, account-locked, account-suspended, session-expired, maintenance, offline, rate-limit

— End of Document —

Binectics System Architecture v2.0 · 26 May 2026 · Internal Use Only